SQL_miniproject_pga58

```sql
-- ============================================================
-- SQL + GenAI Mini Project : Social Media Analytics
-- Dataset : Social_Media
-- Student Name : ____K.GOPICHAND___
-- ============================================================

-- 🚀 SETUP INSTRUCTIONS (MUST DO FIRST)
-- ============================================================
-- Before solving this project, make sure you create and load the dataset.
--
-- STEP 1: Open your SQL client (MySQL Workbench, DBeaver, or SQLite Studio).
-- STEP 2: Run the provided dataset file:
--         social_media_analytics_dataset.sql
--
-- This script will:
--    ✅ Create a new database named `Social_Media`
--    ✅ Create all 7 tables (users, posts, comments, likes, followers, hashtags, post_hashtags)
--    ✅ Insert ~7,000 synthetic rows for analysis
--
-- STEP 3: After successful execution, Your Code the database:
--         USE Social_Media;
--
-- STEP 4: Verify the tables:
--         SHOW TABLES;
--         Your Code COUNT(*) FROM users;
--         Your Code COUNT(*) FROM posts;
--
-- Once you confirm the data is loaded, you can proceed to attempt all project queries.
-- ============================================================

USE Social_Media;

-- ============================================================
-- IMPORTANT: BEFORE USING GenAI FOR QUERY GENERATION
-- ============================================================
-- To help the AI generate accurate SQL, you MUST first share your schema.
-- Paste the following context into ChatGPT (or any GenAI tool) BEFORE you ask your prompts:

/*
You are an expert SQL assistant.
Before answering any question, refer strictly to the database schema provided below.
All SQL queries, joins, and analyses must be based ONLY on this schema — table names, column names, and relationships mentioned here.
Do not assume any extra tables or columns unless explicitly stated.
If a question is ambiguous, clarify it using the schema context rather than inventing new fields.
```

```
Once you understand the schema, wait for my analytical question and generate the most
accurate SQL query for it.

Tables and Key Columns:
  1. users(user_id, username, join_date, country)
  2. posts(post_id, user_id, content, created_at)
  3. comments(comment_id, post_id, user_id, comment_text, created_at)
  4. likes(like_id, post_id, user_id, created_at)
  5. followers(follower_id, user_id, follower_user_id, follow_date)
  6. hashtags(hashtag_id, tag_name, category)
  7. post_hashtags(id, post_id, hashtag_id)

Relationships:
  • Each user can create multiple posts.
  • Each post can have multiple likes and comments.
  • Users can follow each other (self-join in followers table).
  • Posts can be tagged with multiple hashtags (many-to-many via post_hashtags).
*/

-- Once you paste the schema, THEN use prompts like:
--   "Generate SQL to find top 10 active users combining posts and comments."
--   "Find trending hashtags used in more than 20 posts."
-- ============================================================




-- ============================================================
-- Q1. Most Active Users (Posts + Comments)
-- ============================================================
-- Objective : Find top 10 users based on combined number of posts and comments.
-- Example GenAI Prompt :
--   "Write SQL to find top 10 active users combining posts and comments count."
-- Write your query below 👇
-- ------------------------------------------------------------
-- Your Code ...
select u.user_id,u.username,
    count(distinct p.post_id) as total_posts,
    count(distinct c.comment_id) as total_comments,
    count(distinct p.post_id) + count(distinct c.comment_id) as total_activity
from users u
left join posts p
    on u.user_id = p.user_id
left join comments c
    on u.user_id = c.user_id
group by u.user_id, u.username
order by total_activity DESC
limit 10;

-- Solution Summary --
/*This query identifies the top 10 most active users
by combining the number of posts and comments made by each user.
It uses LEFT JOIN to include all users, COUNT(DISTINCT) to avoid duplicate counts,
```

```sql
 GROUP BY to summarize activity per user, and ORDER BY with LIMIT to display the top 10 active users.*/

-- ============================================================
-- Q2. Most Liked Posts and Creators
-- ============================================================
-- Objective : Identify posts with maximum likes along with their creator.
-- Example GenAI Prompt :
--   "Show top 10 posts with most likes and username."
-- ------------------------------------------------------------
-- Your Code ...
select p.post_id,u.username,
    count(l.like_id) as total_likes
from posts p
join users u
    on p.user_id = u.user_id
left join likes l
    on p.post_id = l.post_id
group by p.post_id, u.username
order by total_likes desc
limit 10;

-- Solution Summary --
/* This query finds the top 10 most liked posts along with their creators.
it joins the posts, users, and likes tables, counts the total likes for each post using count(),
groups the results by post, and sorts them in descending order to display the most popular posts. */

-- ============================================================
-- Q3. Top Countries by Average Engagement
-- ============================================================
-- Objective : Find countries with the highest average likes per post.
-- Example GenAI Prompt :
--   "Which countries have highest average likes per post?"
-- ------------------------------------------------------------
-- Your Code ...
select u.country,
    round(count(l.like_id) * 1.0 / count(distinct p.post_id), 2) as avg_likes_per_post
from users u
join posts p
    on u.user_id = p.user_id
left join likes l
    on p.post_id = l.post_id
group by u.country
order by avg_likes_per_post desc;

-- Solution Summary --
/* This query calculates the average likes per post for each country.
 it joins the users, posts, and likes tables, counts the total likes and unique posts,
 calculates the average engagement, and sorts the countries in descending order based on average likes per post.*/


```

```sql
-- ============================================================
-- Q4. Trending Hashtags (Used in >20 Posts)
-- ============================================================
-- Objective : Find hashtags that appear in more than 20 posts.
-- Example GenAI Prompt :
--   "Find hashtags used in more than 20 posts."
-- ------------------------------------------------------------
-- Your Code ...
select h.hashtag_id,h.tag_name,
    count(ph.post_id) as post_count
from hashtags h
join post_hashtags ph
    on h.hashtag_id = ph.hashtag_id
group by h.hashtag_id, h.tag_name
having count(ph.post_id) > 20
order by post_count DESC;

-- Solution Summary --
/* This query counts how many posts use each hashtag and
keeps only hashtags that are used in more than 20 posts.
The results are sorted in descending order based on the number of posts.*/


-- ============================================================
-- Q5. Top Influencers (Users with Most Followers)
-- ============================================================
-- Objective : List users with the highest follower count.
-- Example GenAI Prompt :
--   "Find users with maximum followers."
-- ------------------------------------------------------------
-- Your Code ...
select u.user_id,u.username,
    count(f.follower_user_id) as total_followers
from users u
left join followers f
    on u.user_id = f.user_id
group by u.user_id, u.username
order by total_followers desc
limit 10;

-- Solution Summary --
/*this query finds the top 10 users with the highest number of followers.
 it joins the users and followers tables, counts the followers for each user using count(),
 groups the results by user, and sorts them in descending order of follower count.*/

-- ============================================================
-- Q6. Followers Who Never Interacted
-- ============================================================
-- Objective : Identify users who follow others but have never liked or commented.
-- Example GenAI Prompt :
--   "Show users who follow others but never interacted."
-- ------------------------------------------------------------
-- Your Code ...
```

```sql
select u.user_id,u.username
from users u
join (
    select distinct follower_user_id
    from followers
) f
    on u.user_id = f.follower_user_id
left join (
    select distinct user_id
    from likes
) l
    on u.user_id = l.user_id
left join (
    select distinct user_id
    from comments
) c
    on u.user_id = c.user_id
where l.user_id is null
  and c.user_id is null
order by u.user_id;

-- Solution Summary --
/* This query finds users who follow at least one person
but have never liked or commented on any post.*/


-- ============================================================
-- Q7. Hashtags with Highest Engagement
-- ============================================================
-- Objective : Calculate total engagement (likes + comments) for each hashtag.
-- Example GenAI Prompt :
--   "Calculate engagement score per hashtag."
-- ------------------------------------------------------------
-- Your Code ...
with like_counts as (
    select
        post_id,
        COUNT(*) as likes
    from likes
    group by post_id
),
comment_counts as (
    select
        post_id,
        COUNT(*) as comments
    from comments
    group by post_id
)
select h.hashtag_id,h.tag_name,
    coalesce(SUM(lc.likes), 0) as total_likes,
    coalesce(SUM(cc.comments), 0) as total_comments,
    coalesce(SUM(lc.likes), 0) + coalesce(SUM(cc.comments), 0) as engagement
from hashtags h
join post_hashtags ph
```

```sql
    on h.hashtag_id = ph.hashtag_id
left join like_counts lc
    on ph.post_id = lc.post_id
left join comment_counts cc
    on ph.post_id = cc.post_id
group by h.hashtag_id, h.tag_name
order by engagement DESC
LIMIT 10;

-- Solution Summary --
/* This query calculates hashtag engagement as total likes plus total
 comments and returns the 10 highest-engagement hashtags.*/

-- ============================================================
-- Q8. Busiest Posting Hours or Days
-- ============================================================
-- Objective : Find which hour/day sees most posting activity.
-- Example GenAI Prompt :
--   "Write SQL to show which hour or weekday sees most posts."
-- ------------------------------------------------------------
-- Your Code ...
select
    hour(created_at) as posting_hour,
    COUNT(*) as post_count
from posts
group by hour(created_at)
order by post_count DESC
LIMIT 10;

select
    dayname(created_at) as weekday,
    COUNT(*) as post_count
from posts
group by dayname(created_at)
order by post_count DESC;

-- Solution Summary --
/* These queries count posts by hour and by weekday to
 identify the busiest posting periods.*/

-- ============================================================
-- Q9. Inactive Users
-- ============================================================
-- Objective : Find users who have never posted, liked, or commented.
-- Example GenAI Prompt :
--   "Find users who have never posted, liked, or commented."
-- ------------------------------------------------------------
-- Your Code ...
select u.user_id,u.username
from users u
left join (
    select distinct user_id
    from posts
) p
```

```sql
    on u.user_id = p.user_id
left join (
    select distinct user_id
    from likes
) l
    on u.user_id = l.user_id
left join (
    select distinct user_id
    from comments
) c
    on u.user_id = c.user_id
where p.user_id is null
  and l.user_id is null
  and c.user_id is null
order by u.user_id;

-- Solution Summary --
/* This query finds users with no posts, likes, or comments,
meaning they have never been active on the platform.*/

-- ============================================================
-- Q10. Top Countries with Most Influencers
-- ============================================================
-- Objective : Identify countries with the highest number of influencers.
-- Example GenAI Prompt :
--   "Generate SQL to find countries that have the most followed users."
-- ------------------------------------------------------------
-- Your Code ...
select u.country,
    count(distinct f.user_id) as total_influencers
from users u
join followers f
    on u.user_id = f.user_id
group by u.country
order by total_influencers desc;

-- Solution Summary --
/* this query identifies the countries with the highest number of influencers.
it joins the users and followers tables, counts the users who have followers in each
country,
groups the results by country, and sorts them in descending order.*/

-- ============================================================
-- BONUS CHALLENGES
-- ============================================================
-- 1. Engagement rate = (likes + comments) / posts
select u.user_id,u.username,
    count(distinct p.post_id) as total_posts,
    count(distinct l.like_id) as total_likes,
    count(distinct c.comment_id) as total_comments,
    round(
        (count(distinct l.like_id) + count(distinct c.comment_id))
        / count(distinct p.post_id), 2) as engagement_rate
from users u
```

```sql
join posts p
    on u.user_id = p.user_id
left join likes l
    on p.post_id = l.post_id
left join comments c
    on p.post_id = c.post_id
group by u.user_id, u.username
order by engagement_rate desc;

-- 2. Mutual followers
select f1.user_id,f1.follower_user_id
from followers f1
join followers f2
    on f1.user_id = f2.follower_user_id
   and f1.follower_user_id = f2.user_id;

-- 3. Most used hashtags by top 5 influencers
select h.tag_name,
    count(ph.post_id) as total_usage
from hashtags h
join post_hashtags ph on h.hashtag_id = ph.hashtag_id
join posts p on ph.post_id = p.post_id
join (
    select
        u.user_id
    from users u
    join followers f
        on u.user_id = f.user_id
    group by u.user_id
    order by count(f.follower_user_id) desc
    limit 5
) t
    on p.user_id = t.user_id
group by h.hashtag_id, h.tag_name
order by total_usage desc;

-- 4. Country-wise engagement leaderboard
select u.country,
    count(distinct l.like_id) as total_likes,
    count(distinct c.comment_id) as total_comments,
    count(distinct p.post_id) as total_posts
from users u
join posts p
    on u.user_id = p.user_id
left join likes l
    on p.post_id = l.post_id
left join comments c
    on p.post_id = c.post_id
group by u.country
order by total_likes + total_comments desc;
-- ------------------------------------------------------------

-- ============================================================
-- REFLECTION
```

```
-- ============================================================
-- 1. How did GenAI assist you in solving these queries?
--      genai helped generate sql queries quickly,suggested joins and aggregate functions,
--          and improved query accuracy by reducing syntax errors.

-- 2. What optimization tips did you learn?
--       i learned to use proper joins, group by, having, aggregate functions, and indexing concepts
--                   to improve query performance and readability.

-- 3. What business insights stood out to you?
--        the analysis identified the most active users, top influencers, trending hashtags, highly engaged countries,
--            and the overall user engagement patterns, which can support better business decisions.
-- ============================================================

```